

→ masaischool.com

→ Register for a course.

Authentication

→ Concept using which a website will identify the user.

Authorization

masaischool.com / admin

→ Authentication + Role / Permission

Authentication

→ email + password / otp

→ mobile + password / otp

Authorization

→ Authentication + extra role / permission.

Movie Booking Website

Authentication

Authorization

① See the availability of a show.

X

X

② Book a seat

✓

X

③ Cancel a ticket

✓

✓

RBAC

↳ Role Based Access Control.

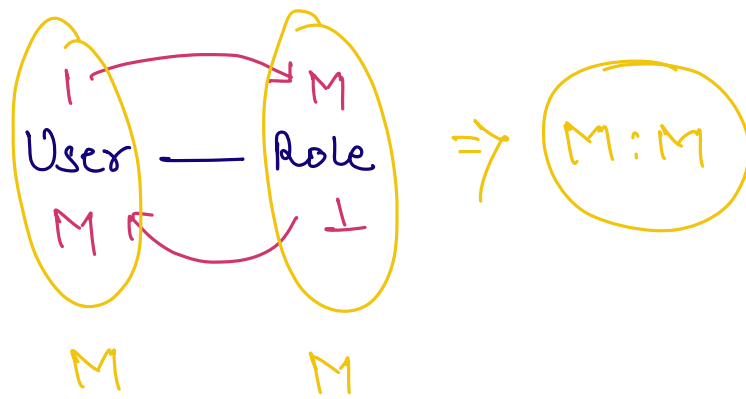
→ Authorization.

users

id	email	name	password

roles

id	name
1	ADMIN
2	MENTOR
3	STUDENT
4	TA
=	=



⇒ Mapping | Lookup Table

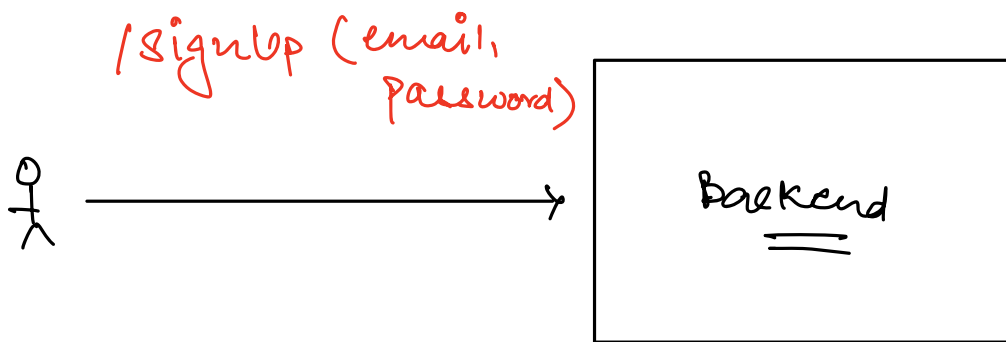
users_roles

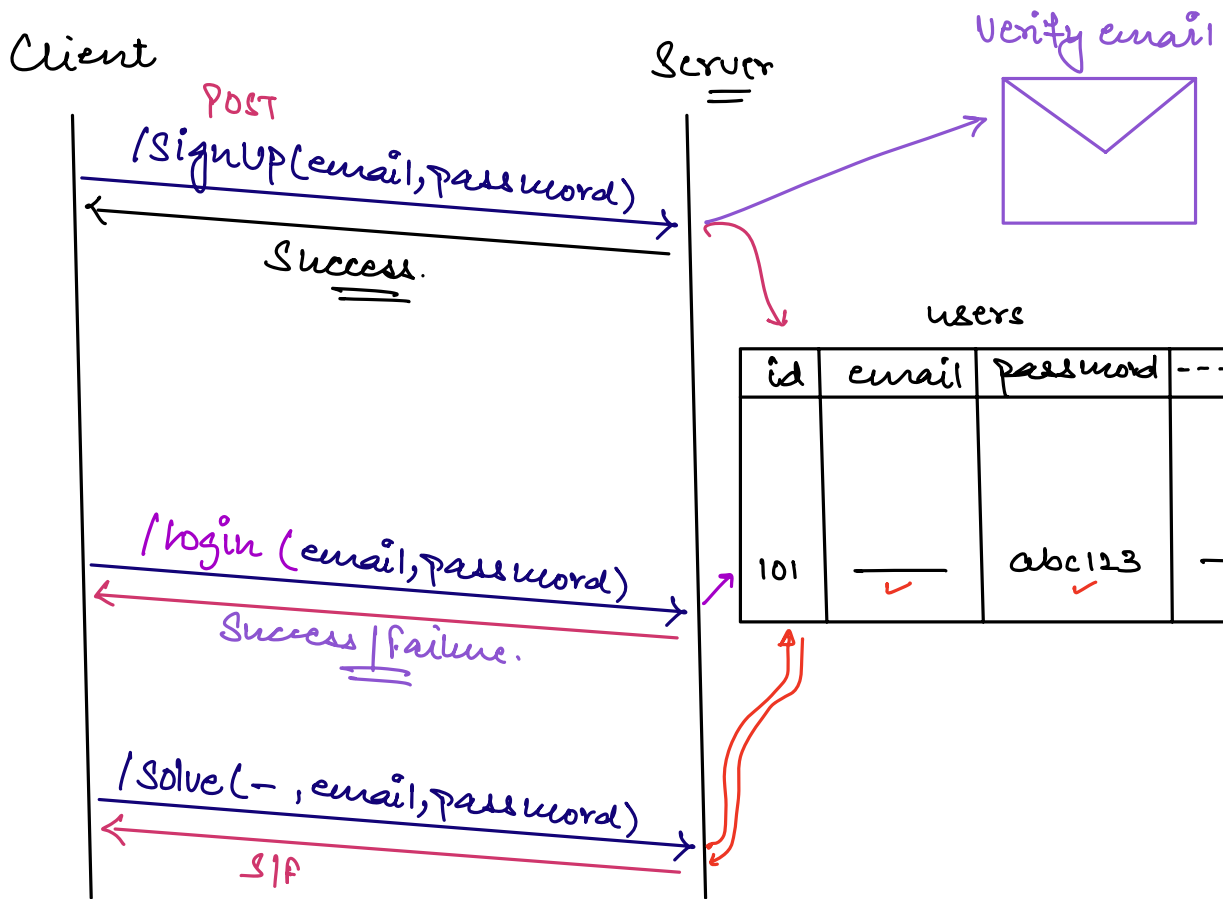
userid	role-id
1	1
1	2
2	3
3	2
3	4

How Authentication Flow works?

→ Sign-Up (Create an Account).

→ email / phone
+
password / otp.





HTTP Requests are stateless.

↳ Every http req. is completely independent of the previous request.

masaischool.com

↳ login(email, password).

→ Because HTTP requests are stateless that's why we need to send the email & password with every request to authenticate.

→ TOKEN.

→ Password shouldn't be stored in the raw format in the DB.

Hashed password.

↳ Hashing.

Input → hash fun[^] → Output

x
x
x
hash() ⇒ same

users

id	email	password	-----
		x6y791α\$	
101	✓	x6y791α\$	_____

abc123 → x6y791α\$

↓
Hacked.

abc@gmail.com } →
abc123

→ Hashing + Salting → Timestamp + ip + mac id +
.....
Randomness.

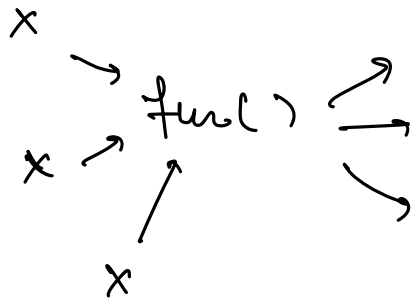
abc123 →

abc123 →

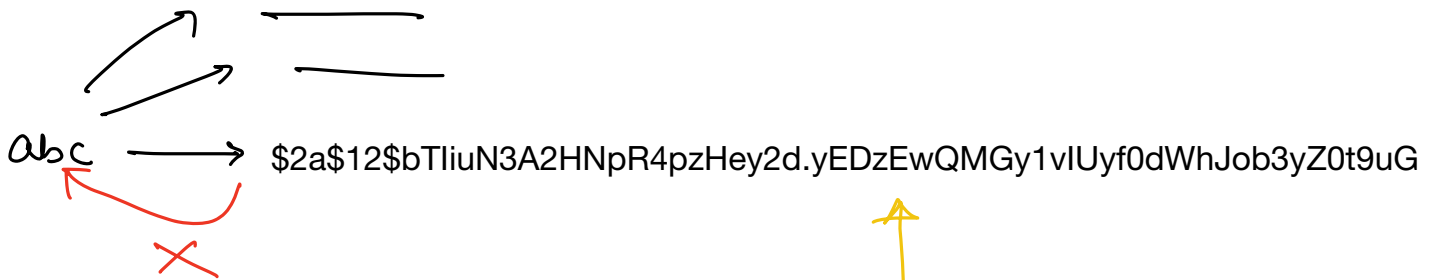
id	email	password	
7	xy2@ —	x\$!@xypl87@	
101	randome —	mox!_@+-178	
109	—	&*C\${acd1257!	

Encrypt Password Encoder

→ This always gives us a different output even for the same input.



→ We can't get the original string back from the encoded or generated string.



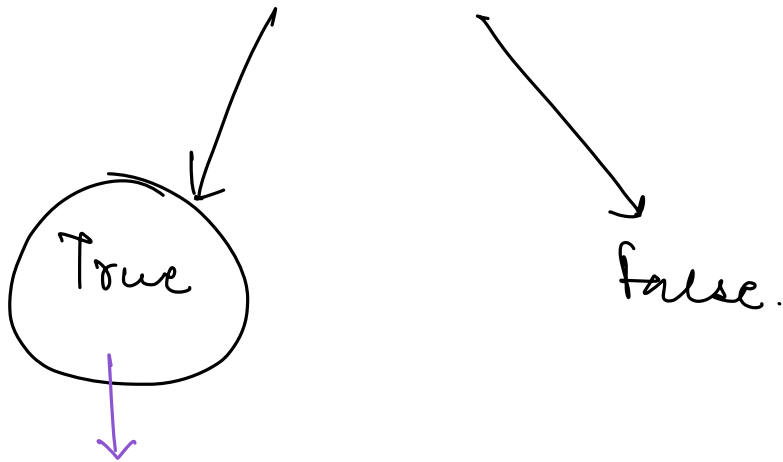
⇒ how do we validate?

signup { deepak@gmail.com
abc

login { deepak@gmail.com
abc

→ matches (rawpassword, encodedPassword)

matches (abc, \$2a\$12\$bTliuN3A2HNpR4pzH
ey2d.yEDzEwQMGy1vIUyf0dW
hJob3yZ0t9uG)



Returns true if
the encoded string
can be generated
using the raw password
or not.

Token.

Client

Server

POST
/signup(email, password)
Success.

/login(email, password)
token

Attempt login (token)
token

Browser stores
the tokens in the
Cache.

users

id	email	password	-----
101	✓	abc123 ✓	_____

tokens

id	user id	value	Expires
1025	101	axyqm121xy	24 th July - 6:55pm.

DB Call to validate
the token.

↓
DB Call is a n/w call.
& can be slow.

amazon.in : —

scaler : —

masai : —

fb : —

x : —

JWT.

↳ JSON Web Token.

→ Self Validating Token.